

Plan de Trabajo Formal - Proyecto Final: BitQuest

Curso: Lenguaje Ensamblador
Proyecto: BitQuest: Explorador de Matrices con C y NASM (64 bits)
Fecha de Entrega Interna: 14 de junio de 2026
Fecha de Entrega Oficial: 15 de junio de 2026

1. Descripción General del Proyecto

El proyecto consiste en el desarrollo de un videojuego en consola utilizando programación híbrida (C y NASM de 64 bits). El juego representa un laberinto mediante una matriz de 60x60, de la cual solo se mostrará una ventana visible de 20x20 en pantalla. El jugador (P) deberá moverse, recolectar monedas (M), buscar una llave (K), abrir una puerta (D) y llegar a la salida (E) para avanzar de nivel.

2. Estructura del Repositorio en GitHub

Para asegurar un desarrollo organizado y un historial de commits progresivo, el repositorio se estructurará de la siguiente manera:

- **src/:** Contiene los archivos de código fuente (main.c, juego.c, rutinas.asm).
- **include/:** Archivos de cabecera (juego.h, mapas.h).
- **assets/:** Archivos de texto para los mapas (nivel1.txt, nivel2.txt, etc.) y recursos de audio o música.
- **build.bat:** Script de compilación automatizada para Windows.
- **build.sh:** Script de compilación automatizada para Linux (entornos virtuales como Ubuntu).
- **README.md:** Instrucciones claras de compilación y ejecución del videojuego.

3. Distribución de Roles y Responsabilidades

A continuación se detalla la asignación de tareas para cada integrante del equipo, diseñada para cubrir los requisitos obligatorios y los puntos extra planificados de forma equilibrada.

Integrante	Rol Principal	Tareas Específicas y Entregables
Aída (Líder)	Dirección, DevOps, Arte Visual y Audio	• Diseño lógico de los 4 mapas de 60x60 integrando bloques ASCII. • Implementación de la interfaz de color en la

Integrante	Rol Principal	Tareas Específicas y Entregables
		consola mediante secuencias de escape ANSI. • Desarrollo de la pantalla de inicio (selector de niveles 1 a 4) y la pantalla de victoria final. • Integración del sistema de audio (música de fondo y efectos de sonido para monedas, llaves y victoria). • Gestión integral de las ramas en el repositorio de GitHub y resolución de conflictos de código. • Creación de los scripts de compilación automatizada (build.bat y build.sh). • Redacción del reporte de laboratorio final en PDF y control de calidad general.
Leo	Desarrollador del Motor del Juego (C)	• Programación de la lógica del Viewport dinámico de 20x20 basado en la posición del jugador. • Captura de comandos de entrada del teclado (W, A, S, D) y la tecla de salida segura (Q). • Estructuración de la interfaz en C para realizar el paso de parámetros y la recepción de retornos desde NASM.
Beto	Desarrollador de Bajo Nivel (NASM - Módulo A)	• Función Obligatoria 1: Contar caracteres

Integrante	Rol Principal	Tareas Específicas y Entregables
		específicos dentro del mapa para calcular en tiempo de ejecución las monedas totales de cada nivel. ● Función Obligatoria 2: Validar movimiento (verificación de colisiones contra las paredes marcadas con el carácter #).
Arthur	Desarrollador de Bajo Nivel (NASM - Módulo B)	● Función Obligatoria 3: Implementación del cálculo del puntaje matemático final cruzando variables de monedas, pasos y niveles. ● Función Obligatoria 4: Detectar la presencia exacta de un objeto (K, D, E, M) en coordenadas específicas. ● Función Obligatoria 5: Conteo completo de las celdas marcadas como camino libre dentro del mapa.
Carlos	Desarrollador de Persistencia y Transición (C)	● Desarrollo del analizador (parser) en C para leer y cargar los mapas desde archivos planos .txt externos. ● Programación de las pantallas de resumen intermedio al terminar cada nivel (monedas del nivel y pasos). ● Controlador de transiciones lógicas entre niveles enlazado

Integrante	Rol Principal	Tareas Específicas y Entregables
		al menú de inicio de la líder.

4. Cronograma de Desarrollo Diario (8 al 14 de Junio)

Fecha	Objetivos y Entregables Clave	Asignados
8 de Junio	Estructuración del repositorio en GitHub, definición formal de los prototipos de función y diseño inicial de los mapas base.	Aída, Carlos
9 - 10 de Junio	Desarrollo del Viewport de 20x20 en C. Programación de las funciones de validación de movimiento y conteo de monedas en NASM.	Leo, Beto, Arthur
11 de Junio	Implementación del sistema de carga de archivos .txt en C. Codificación de las funciones restantes en NASM (puntaje y celdas libres).	Carlos, Arthur
12 de Junio	Integración general: combinación de menús, aplicación de colores ANSI, renderizado de bloques ASCII y llamadas al sistema de sonido.	Aída, Leo, Carlos
13 de Junio	Fase intensiva de pruebas funcionales cruzadas tanto en entornos Windows como en Linux (máquina virtual Ubuntu). Corrección de bugs.	Todo el equipo
14 de Junio	Congelamiento de código fuente (Code Freeze), validación final del historial de commits y armado del reporte PDF explicativo.	Aída

5. Pautas de Código y Estilo de Programación

Para asegurar una correcta presentación con el profesor Irving, el código debe seguir un estilo claro, progresivo y completamente documentado. Se debe priorizar que la lógica demuestre el entendimiento fundamental de los principios del lenguaje, evitando estructuras excesivamente complejas u optimizaciones oscuras que oculten el proceso de aprendizaje del estudiante.

Ejemplo Base de Estructura NASM (64 bits)

```
;
-----
-----
; rutinas.asm - Funciones en Lenguaje Ensamblador de 64 bits
;
-----
-----
global contar_caracteres

section .text

; Función: contar_caracteres
; Parámetros (Convención de llamadas en sistemas de 64 bits):
; RDI = Dirección base de la matriz de caracteres (mapa)
; RSI = Número total de celdas en memoria (filas * columnas)
; RDX = Carácter objetivo a buscar (contenido en el registro bajo DL)
contar_caracteres:
    push rbp
    mov rbp, rsp

    xor rax, rax          ; Inicializar acumulador de coincidencias
    (retorno)
    xor rcx, rcx          ; Inicializar índice del bucle

.bucle_inicio:
    cmp rcx, rsi
    jge .bucle_fin

    mov bl, byte [rdi + rcx]
    cmp bl, dl
    jne .avanzar_indice

    inc rax                ; Se incrementa al hallar coincidencia exacta

.avanzar_indice:
    inc rcx
    jmp .bucle_inicio
```

```
.bucle_fin:
    mov rsp, rbp
    pop rbp
    ret
```

Archivo de Inclusión Técnico de Referencia (juego.h)

```
#ifndef JUEGO_H
#define JUEGO_H

// Declaración de las 5 funciones obligatorias en NASM
extern long long contar_caracteres(char* mapa, int total_celdas, char
caracter);
extern int validar_movimiento(char* mapa, int columnas, int
nueva_fila, int nueva_col);
extern long long calcular_puntaje(int monedas, int pasos, int
niveles);
extern int detectar_objeto(char* mapa, int columnas, int fila, int
col, char objeto);
extern long long contar_celdas_libres(char* mapa, int total_celdas);

// Funciones del sistema de juego controladas en C
void cargar_mapa_desde_archivo(const char* ruta_archivo);
void renderizar_ventana_visible(int pos_x, int pos_y);
void desplegar_pantalla_resumen(int nivel_actual, int monedas, int
total_monedas, int pasos_dados);

#endif
```

6. Criterios Importantes de Calidad y Penalizaciones

- Queda estrictamente prohibido asignar el número de monedas mediante variables fijas o constantes en el código de C. El total debe ser calculado recorriendo la memoria con la función en ensamblador.
- El historial del repositorio de GitHub debe reflejar la evolución real y progresiva del proyecto a través de múltiples confirmaciones (commits). Un repositorio con un único commit final será fuertemente penalizado.
- La correcta validación de límites de la matriz y colisiones con obstáculos (#) debe operar de manera perfecta antes de dar paso a las características de personalización estética.